

中山大学计算机学院本科生实验报告

(2024 学年第 1 学期)

课程名称: Data structures and algorithms

任课教师: 张子臻

年级	23 级	专业 (方向)	计算机科学与技术
学号	23320093	姓名	林宏宇
电话	13421145433	Email	linhy69@mail2.sysu.deu.cn
开始日期	2024.10.11	完成日期	2024.10.11

第一题

1. 实验题目

Can I Post the letter

2. 实验目的

I am a traveler. I want to post a letter to Merlin. But because there are so many roads I can walk through, and maybe I can't go to Merlin's house following these roads, I must judge whether I can post the letter to Merlin before starting my travel. Suppose the cities are numbered from 0 to $N-1$, I am at city 0, and Merlin is at city $N-1$. And there are M roads I can walk through, each of which connects two cities. Please note that each road is direct, i.e. a road from A to B does not indicate a road from B to A . Please help me to find out whether I could go to Merlin's house or not.

3. 算法设计

- 1.利用矩阵 graph 表示 cities 之间存在 road, 并且在这里 road 为有向的, 矩阵并非对称的
- 2.走路函数 Go, 从城市 0 开始, 如果有通往下一个城市的 road 则接着 Go, Go 的终止条件为当走到了 $N-1$ 的城市, 并且将结果 true 一级一级传回原来的起始递归位置, 如果找不到接下来的路则自动回到调用递归的上一级位置, 寻找下一条路
- 3.为了防止进入死循环, 如从 0 走到 1, 再从 1 走到 0, 需要使用 visited 数组, 再调用 Go 函数的开始进行标记此时所在 city, 表示某个结点已经访问过, 不再从这一条 road 走
- 4.当 0 的所有 road 都遍历完全时, 还没有找到结果, 则返回失败。

4. 程序运行与测试

1. main 函数不断循环，当输入案例的第一个输入为 0 时则退出

```
while(1)
{
    int N;//citis
    cin>>N;
    if(N==0)
        return 0;//结束的标志 case的第一个输入为0
```

2. 在 main 函数里创建矩阵 graph、数组 visited，并对他们进行初始化和赋值

```
vector<vector<int>> graph(N,vector<int>(N));
while(M--)
{
    int a,b;
    cin>>a>>b;
    graph[a][b]=1;//有路可走
}
vector<bool> visited(N,0);
```

- 3 在 main 里调用 Go 函数，判断输出

```
if(Go(0,graph,N,visited))
    cout<<"I can post the letter"<<endl;
else
    cout<<"I can't post the letter"<<endl;
}
```

3. Go 函数的实现

```
bool Go(int begin,vector<vector<int>>& graph,int N,vector<bool>& visited)
{
    if(begin==N-1)
        return true;
    visited[begin]=1;
    for(int i=0;i<N;i++)
    {
        if(graph[begin][i]==1&&visited[i]==0)
        {
            if(Go(i,graph,N,visited))
                return true;
        }
    }
    return false;
}
```

5. 实验总结与心得

- 1.实验题相对简单，只需要熟悉矩阵表示图、进行递归调用即可
- 2.易错点在于容易进入死循环，需要使用标记
- 3.此题图为有向图，与其他题目的无向图有些不同
- 4.需要设计好一直走路的函数，注意考虑是否有递归中的回溯

第二题

1. 实验题目

Compute Active time Interval during DFS

2. 实验目的

Problem

Given an undirected graph, for every vertex, compute the starting time and finishing time during a DFS.

Input

The first line is the number of test cases.

For every test case, the first line is the number of node n , meaning nodes are $1, 2, \dots, n$.

The next line is the number of edges m , then m lines are followed, where each line is in the form of $u \ v$, meaning (u, v) is an undirected edge.

Output

For each test case, print the starting and finishing time for every vertex, suppose we always starting a DFS from the smallest numbered vertex. For example, the output might look like

```
1:1-2
2:3-4
3:5-6
---
```

1. Suppose we always start a DFS from the smallest numbered vertex, and choose the smallest numbered adjacent vertex.
2. The starting time is 1.
3. Print a dash-line after each test case.

3. 算法设计

- 1.创建二位数组 graph 作图的邻接表
- 2.创建 valid 用来标记某个结点是否访问过
- 3.创建结果数组 result，用来记录每个结点访问的开始时间和结束时间
- 4.DFS 函数：①利用栈来遍历图 ②将第一个起始压入栈，入栈时记录 result 该结点的起始时间 ③获取栈顶结点，并将结点出栈 ④遍历此结点的所有邻居，如果邻居未被访问，将当前结点入栈，再将邻居入栈，并记录邻居的 result ⑤如果当前结点的所有邻居结点访问完了，则作时间记录 result ⑥循环上述直到栈为空

4. 程序运行与测试

1. main 函数的输入输出，graph、result、valid 创建，dfs 函数调用

```

int case_nums;
cin >> case_nums; // 输入测试用例的数量

while (case_nums--)
{
    int node_nums;
    cin >> node_nums; // 输入节点数量
    int edse_nums;
    cin >> edse_nums; // 输入边的数量

    // 初始化邻接表和访问标记数组
    vector<vector<int>> graph(node_nums + 1); // 图的邻接表, 节点编号从1到node_nums
    vector<int> valid(node_nums + 1, 0); // 标记数组, 0表示未访问, 1表示已访问

    // 输入图的边
    while (edse_nums--)
    {
        int a, b;
        cin >> a >> b;
        graph[a].push_back(b);
        graph[b].push_back(a); // 无向图的边
    }

    // 初始化结果数组, 记录每个节点的开始时间和结束时间
    vector<vector<int>> result(node_nums + 1, vector<int>(2, 0));

    int Time = 1; // 时间戳从1开始
    for (int i = 1; i <= node_nums; i++)
    {
        if (valid[i] == 0) // 如果节点i没有被访问过
            dfs(i, graph, result, valid, Time); // 从该节点开始DFS
    }

    // 输出每个节点的开始和结束时间
    for (int i = 1; i <= node_nums; i++)
    {
        cout << i << ":" << result[i][0] << "-" << result[i][1] << endl;
    }
    cout << "----" << endl; // 每个测试用例之间的分隔线
}

```

2.dfs 函数的实现

```

void dfs(int start, vector<vector<int>>& graph, vector<vector<int>>& result, vector<int>& valid, int& Time)
{
    stack<int> s;
    s.push(start); // 将起始节点压入栈中
    valid[start] = 1; // 标记起始节点为已访问
    result[start][0] = Time++; // 记录起始节点的开始时间

    while (!s.empty())
    {
        int node = s.top(); // 获取栈顶节点
        s.pop();

        bool all_neighbors_visited = true; // 用来检查当前节点的所有邻接节点是否都访问过

        // 遍历当前节点的所有邻接节点
        for (int neighbor : graph[node])
        {
            if (!valid[neighbor]) // 如果邻居未被访问
            {
                valid[neighbor] = 1; // 标记邻居为已访问
                result[neighbor][0] = Time++; // 记录邻居的开始时间
                s.push(node); // 将当前节点重新压入栈中，继续遍历
                s.push(neighbor); // 将邻居节点压入栈中，继续遍历
                all_neighbors_visited = false; // 当前节点有未访问的邻居
                break; // 找到一个未访问的邻居就继续
            }
        }

        // 如果当前节点的所有邻接节点都已访问，记录当前节点的结束时间
        if (all_neighbors_visited)
        {
            result[node][1] = Time++; // 记录当前节点的结束时间
        }
    }
}

```

5. 实验总结与心得

- 1.实验题目最大的难点是理解题意，明白何时记录 starting 和 ending，在入栈的时候记录 starting，并在所有邻居结点访问完的时候记录 ending，并且操作依次使用 Time
- 2.根据逻辑实现 dfs 函数，在这一题实现的函数里，当访问的结点邻居完了则记录 ending，还没完则有对应的操作，要将当前结点返回栈，同时将邻居加入栈，遵守优先深度遍历

第三题

1. 实验题目

Connect components in undirected graph

2. 实验目的

1. 题目描述

输入一个简单无向图，求出图中连通块的数目。

输入描述

输入的第一行包含两个整数 n 和 m ， n 是图的顶点数， m 是边数。 $1 \leq n \leq 1000$ ， $0 \leq m \leq 10000$ 。

以下 m 行，每行是一个数对 $v\ y$ ，表示存在边 (v,y) 。顶点编号从 1 开始。

输出描述

单独一行输出连通块的数目。

3. 算法设计

1. 创建二维数组图的邻接表 `graph`、创建访问记录 `valid`

2. `dfs` 函数，`dfs` 时，目的是将块上所有结点的 `valid` 都记录一下，然后在 `main` 里 `count++`

3. 当 `main` 里所有结点都遍历完了即可输出 `count`

4. 程序运行与测试

1. `main` 函数

```

int main()
{
    int n,m;
    cin>>n>>m;
    vector<vector<int>> graph(n+1);
    vector<int> valid(n+1,0);
    while(m--)
    {
        int a,b;
        cin>>a>>b;
        graph[a].push_back(b);
        graph[b].push_back(a);
    }
    int count=0;
    for(int i=1;i<=n;i++)
    {
        if(valid[i]!=1)//还未被访问
        {
            dfs(graph,i,valid);
            count++;
        }
    }
    cout<<count<<endl;

    return 0;
}

```

2. dfs 函数


```

void dfs(vector<vector<int>>& graph,int i,vector<int>& valid)
{
    stack<int> s;
    s.push(i);
    valid[i]=1;
    while(!s.empty())
    {
        int node=s.top();
        s.pop();
        for(int i=0;i<graph[node].size();i++)
        {
            if(valid[graph[node][i]]!=1)
            {
                s.push(graph[node][i]);
                valid[graph[node][i]]=1;
            }
        }
    }
}

```

5. 实验总结与心得

- 1.该实验难度不大，要明白如何记录块数：每一次 dfs 结束即记录一次块
- 2.作 valid 标记表示是否已经遍历过该节点
- 3.dfs 使用深度优先遍历，配合栈的使用，目的是为了遍历对块上所有结点做记录。